SENIOR PROJECT TECHNICAL PROPOSAL

Zambeel Student Agentic Harness

LLM orchestration, retrieval-augmented policy reasoning,
and governed ERP workflows

Muhammad Hussain Baqar | Muhammad Yahya Rahim | Muhammad Ammar Anees
Saad Jamshaid Khan | Nausherwan Khan

Proposal. Students at LUMS currently have no unified way to navigate academic processes. A single request—force enrollment, TA-ship application, or course registration—requires the student to manually interpret policy documents, check Zambeel, email a professor, follow up for status, and wait on an approval chain they have no visibility into. We propose an agentic harness over Zambeel that accepts natural language requests, retrieves the governing policy, enforces role-based approval workflows, and executes only authorized actions—reducing a multi-day, multi-email process to a single interaction.

1 Problem, Objective and MVP Boundary

The MVP is deliberately centered on one demonstrable, approval-based workflow.

- **Primary MVP workflow:** force enrollment, end to end—from the student's request to professor approval, Registrar processing and automated notification.
- **Secondary workflows:** TA-ship applications and approvals, prerequisite/seat/clash checks, request status tracking and degree-progress review.
- **Primary actors:** student, instructor/HOD, Registrar and authorized Zambeel services.
- **Out of scope:** autonomous grade changes, policy exceptions, disciplinary decisions, financial transactions and unrestricted database access.

TA-ship follows the same approval pattern as force enrollment—student application, instructor nomination and department approval—and reuses the same governed workflow infrastructure.

System context. Zambeel is LUMS's ERP and student information system, handling course registration, enrollment records and academic workflows. The harness does not replace Zambeel; it sits in front of it, accepting natural language requests and executing only through narrow, authorized adapters. During development, a mock Zambeel API built on synthetic data stands in for the real system.

2 Use-Case View

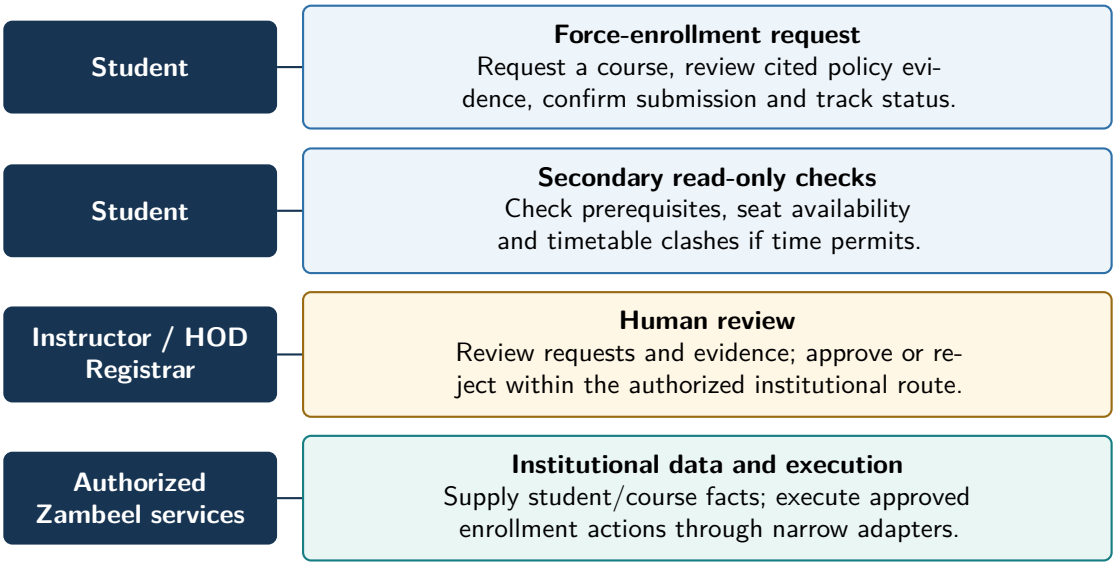


Figure 1: Actor-to-capability view. Each line associates an actor with its capabilities; rows are not a sequence. Consequential actions require policy validation, student confirmation and human approval.

Primary flow—Force Enrollment

1. The student tells the agent: "I want to request force enrollment in CS 412 next semester."
2. The agent extracts the intent, checks the student's profile, and verifies prerequisites and seat status through read adapters.
3. The agent retrieves the governing force-enrollment policy section from the RAG store and cites it in the response.
4. The student confirms they want to proceed; the system creates a `WORKFLOW_CASE` record.
5. The workflow routes an approval request to the relevant instructor.
6. The instructor uses the agent interface to approve or reject the request. For approval, the system verifies the instructor's role and generates a one-time signed approval token.
7. The workflow validates the token, transitions the case to `APPROVED`, and calls `execute_approved_enrollment`.
8. The mock Zambeel service processes the enrollment and the student receives an automated notification.
9. The system stores a complete audit trail: actor, timestamp, policy version, approval decision and tool result.

3 Mock Harness Design: LLM + RAG

The LLM plans and coordinates; deterministic services enforce identity, data access, policies, workflow state, approvals and writes. Force enrollment, TA-ship approvals, academic checks, degree-progress review, status lookup and notifications are bounded tools, not independent agents.

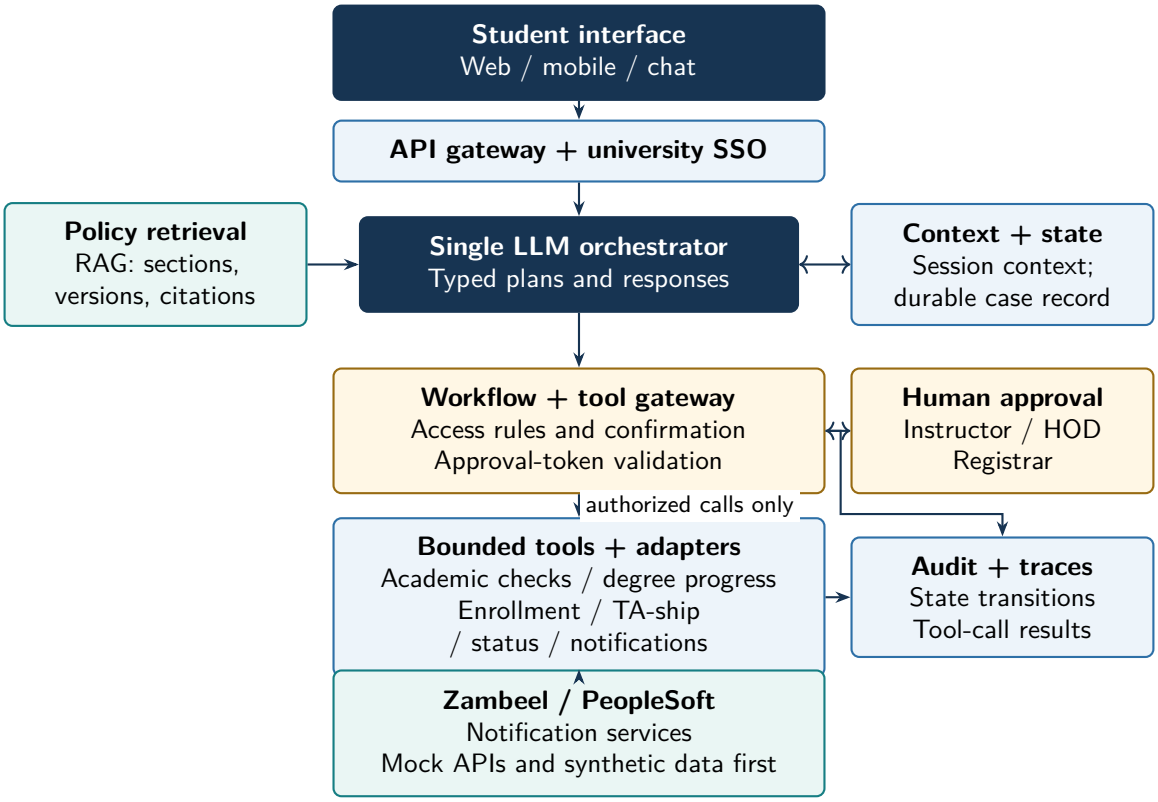


Figure 2: Request and control flow without a specialist-agent layer. Arrows indicate the principal interactions; routine return paths are omitted. The LLM has no direct ERP write access. Policy ingestion feeds RAG; LMS and Outlook adapters are later extensions.

How a request runs

Stage	Harness behaviour
1. Understand	Extract intent and missing fields, then create a typed plan.
2. Ground	Retrieve policy sections with version and source; fetch live student/course facts through authorized read adapters.
3. Check	Invoke the bounded tool required by the selected workflow; prepare the response or case update.
4. Govern	Gate each consequential operation using access controls, rules, confirmation and approval tokens.
5. Execute	Call an idempotent Zambeel tool, verify the result, notify the student and record the full trace.

An approval token is a one-time signed identifier generated when an authorized human approves a request. The workflow engine requires it before calling any write adapter. Without a valid token, execution cannot proceed regardless of what the LLM returns.

Role-Based Access Control

Actor	Can read	Can write / approve
Student	Own profile, own cases, course listings and policy excerpts	Submit requests and confirm own actions
Instructor / HOD	Requests assigned to them for approval	Approve or reject force enrollment for their courses
Registrar	All submitted and approved cases	Authorize or execute enrollment actions in Zambeel
System adapter	Authorized data only through narrow read tools	Write only with a valid approval token and idempotency key

4 Data and Persistence Design

A durable case database is required because approvals and system updates may continue long after the original chat session ends. The ERD is split into three views so that labels and relationships remain legible.

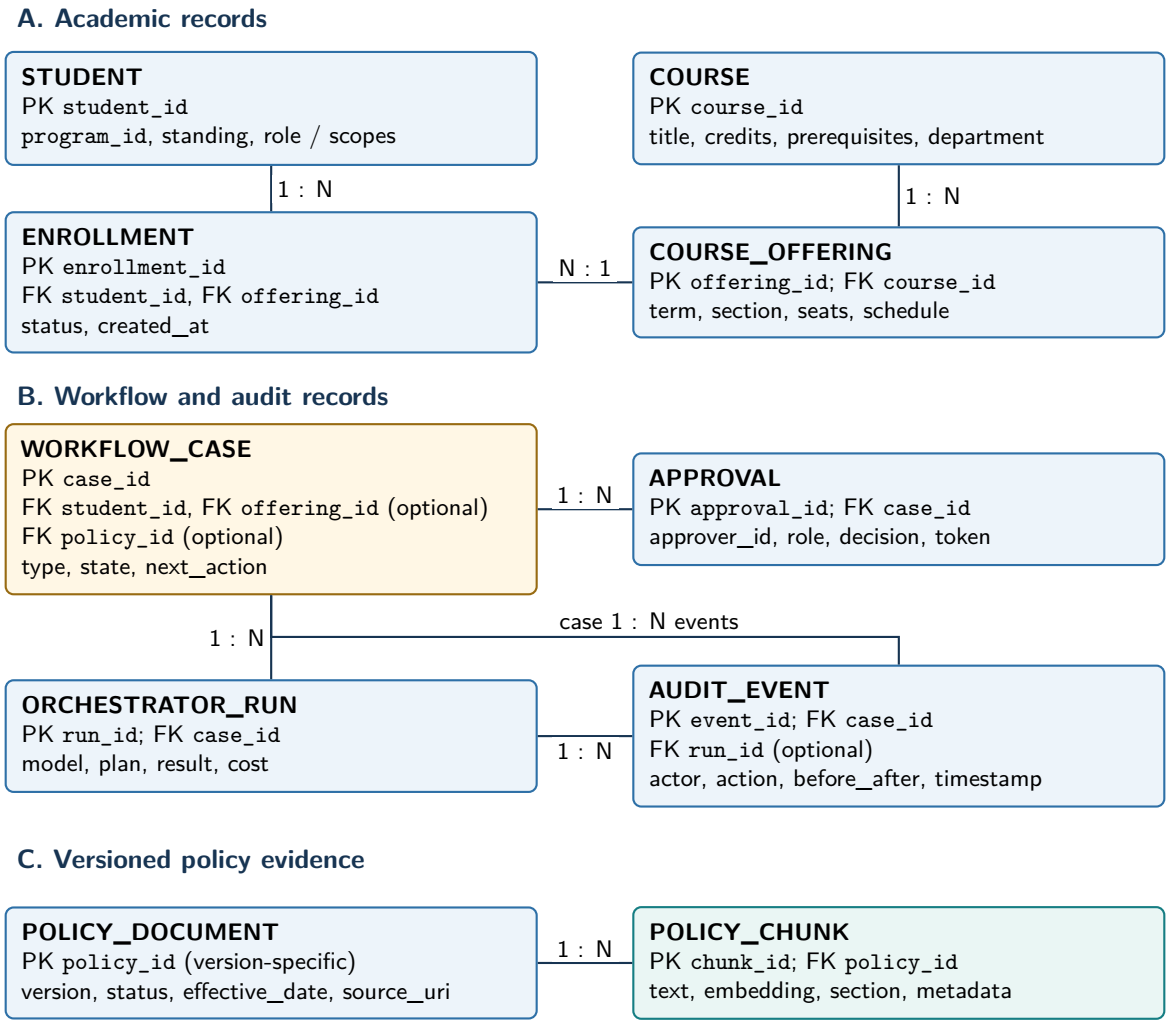


Figure 3: Minimum viable ERD. PK = primary key; FK = foreign key. Read relationships left to right or top to bottom; N denotes many possible child records, not a required minimum. Optional FKs may be null.

Workflow state machine

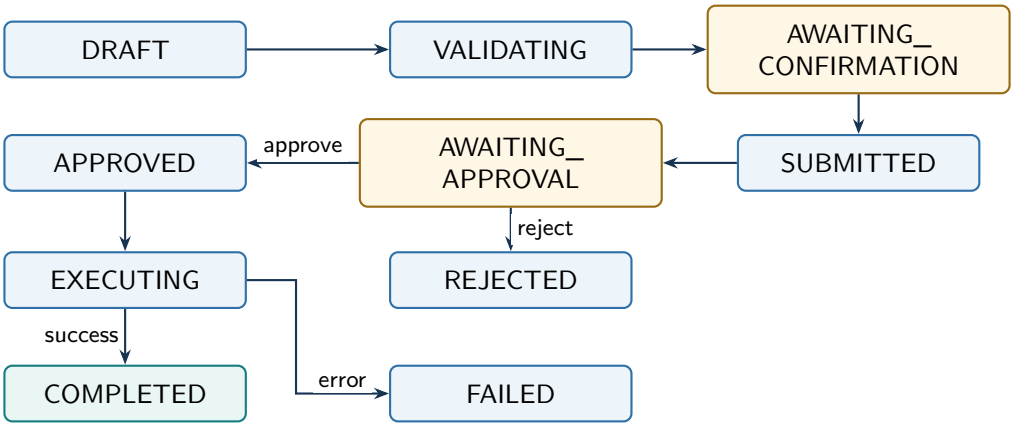


Figure 4: Approval-required write path. Rejection never leads to execution. Failed validation or missing confirmation blocks progression; read-only answers do not enter the write path.

Every transition stores actor, timestamp, reason, policy version and the tool result when applicable. Execution requires valid authorization and an idempotency key; retries must not duplicate enrollment actions.

RAG and memory rules

- Ingest only approved policies; split by section; attach document version, effective date, authority and access level; create embeddings; retrieve hybrid semantic and keyword results.
- The final answer must cite the retrieved policy section. If evidence is missing or contradictory, the harness escalates instead of inventing a rule.
- Conversation memory stores convenience context; `WorkflowCase` remains the authoritative record. Sensitive student data is encrypted, minimized and never placed in the vector index unless approved.

Narrow tool contracts

Read: `get_student_profile`, `search_offerings`, `check_prerequisites`, `check_seats`, `check_clashes`, `get_case_status`, `degree_audit`.

Write: `create_case`, `submit_for_approval`, `execute_approved_enrollment`, `record_taship_decision`, `notify_student`.

5 Implementation Blueprint

Build against synthetic data and a mock Zambeel API first, then replace adapters with supported institutional APIs without redesigning the harness.

Layer	Proposed implementation
Interface	Responsive web chat and structured cards for checks, evidence, confirmation, approval status and citations.
Orchestration	Single orchestrator with typed plans, bounded tool registry, short-term context and durable case state.
RAG	Policy ingestion, hybrid retrieval, metadata filtering, reranking and source-linked responses.
Backend	REST/GraphQL service, workflow engine, PostgreSQL case store, vector index, background jobs and notification queue.
Integration	Mock/authorized Zambeel/PeopleSoft adapters, policy repository, approval portal and email/push; LMS and Outlook as later extensions.
Security	University SSO, RBAC/ABAC, least-privilege service accounts, secrets vault, encryption, rate limits and immutable audit logs.

Team Ownership

The initial ownership split assigns one accountable lead to each major layer; the team can still review and support work across boundaries.

Layer	Owner
Orchestrator and agent logic	Muhammad Hussain Baqar
RAG and policy ingestion	Muhammad Yahya Rahim
Frontend / chat interface	Muhammad Ammar Anees
Workflow engine and database	Saad Jamshaid Khan
Mock Zambeel adapter and integration	Nausherwan Khan